



OOP - Object Oriented Programming

By PrudviKrishna



Procedural Vs OOPs

Procedural

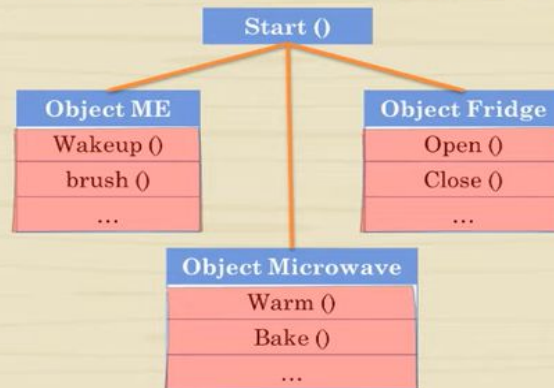
Program is written as sequence of instructions

e.g.: Recipes,
Morning Steps:

Start ()
I wake up
I brush my teeth
I open fridge
I take milk
I warm it in microwave

Object Oriented

Program is an interaction of functions between objects



Procedural

Program is written as sequence of instructions

1. Top down approach
2. It doesn't have proper way for hiding data
3. Data is not secure
4. Code is interdependent
5. Reuse difficult

Object Oriented

Program is an interaction of functions between objects

1. Bottom up approach
2. Helps in wrapping data and functions in a class
3. Helps Building secure programs
4. Code is modular
5. Can be extended for reuse

What is OOPs

Object-Oriented Programming (OOP) is a programming paradigm that revolves around objects and classes. OOP allows us to model real-world entities as objects that contain both data (attributes) and behaviors (methods). It emphasizes reusability, modularity, and the abstraction of complex problems into simpler units.

Key concepts of OOP:

Encapsulation: Bundling data and methods into a single unit (class) and restricting access to some of the object's components.

Inheritance: Allowing one class to inherit properties and methods from another class.

Abstraction: Hiding the complex implementation details and showing only the essential features of an object.

Polymorphism: The ability to define methods in different ways depending on the object's class.



Class and Object

CLASS

- Blueprint for objects
- Defines object behavior

1. **Attributes:** Data/Properties
2. **Methods:** Functions/Behavior

Elements defined within a class are referred to as **members**, which include both attributes and methods.

OBJECT

Object is an instance of a **Class**

Class is the data type;

Object is a variable of a **Class**

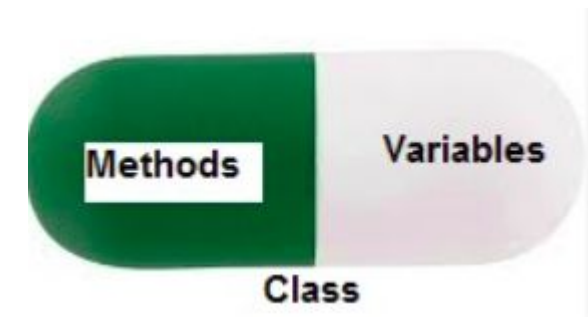


Encapsulation

Encapsulation is an idea of wrapping data and the methods that work on data within one unit.

Example:

```
class BasicClass:  
    name="NareshIT"  
  
    def print_something(self): # donot worry about the self, will discuss  
        print("hello intro to class")  
        print(self.name)  
  
*****  
bc=BasicClass() # creation of Object  
bc.print_something() # calling the method of a class using object.
```





What is Self?

self is a convention used in object-oriented programming to represent the instance of a class. When defining methods in a class, self refers to the object that is calling the method.

Key Points about **self**:

Refers to the Instance: self allows you to access the instance variables (attributes) and methods of the object. It refers to the current instance of the class inside which a method is being called.

Not an Explicit Parameter: Although self is passed as the first parameter to instance methods, it doesn't need to be explicitly passed when calling a method. Python does that behind the scenes.

Not a Keyword: self is not a reserved keyword in Python; it's just a strong convention. You can technically name it anything, but it's strongly recommended to use self for readability and convention.



What is a Constructor?

A constructor is a special method used for initializing newly created objects. It is automatically called when a new object is created from a class, and it allows you to set initial values for the object's attributes.

Syntax : `def __init__(self, ...):`

with out Constructor

```
class BasicClass:
    name="class attribute"

    def print_something(self):
        print("Intro to class")
        print(self.name)

    def assignname(self,newname):
        self.name=newname

bc1=BasicClass()

bc1.assignname("Prudvi")

bc1.print_something()
```

With Constructor

```
class BasicClass:
    name= "class attribute"

    def print_something(self):
        print("Intro to class")
        print(self.name)

    def __init__(self,newname):
        self.name= newname
        self.age = 10

bc1=BasicClass("Prudvi")

bc1.print_something()
```


Instance Method & Attributes

An **instance method** operates on an instance of a class and has access to the instance itself.

Characteristics:

- The first parameter is always self, which refers to the specific instance of the class.
- Can access and modify instance attributes.
- Can call other instance methods of the same object.

```
class Dog:
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age   # Instance attribute

    # Instance method
    def bark(self):
        print(f"{self.name} is barking!")

    # Another instance method
    def birthday(self):
        self.age += 1 # Modify instance attribute
        print(f"Your dog {self.name}, now at the age of {self.age}!")
*****

# Creating an instance of Dog
dog = Dog("Buddy", 5)
dog.bark() # Instance method call
dog.birthday() # Accessing and modifying instance attribute via instance method
```


Class Method & Attributes

A **class method** operates on the class itself rather than on an instance. This means it does not deal with individual object instances, but with the class as a whole.

Characteristics:

- It is marked using the @classmethod decorator.
- The first parameter is always cls, which refers to the class itself (not the instance).
- It can access or modify class attributes

class Dog:

 # Class attribute (shared by all instances)
 species = "Canis familiaris"

 def __init__(self, name):
 self.name = name # Instance attribute

 # Class method

 @classmethod

 def species_info(cls):
 return f"All dogs are of species {cls.species}"

Accessing class method

print(Dog.species_info()) # Can be called on the class itself, **no instance required**

static Method

static method is a method that belongs to a class rather than to an instance of the class.

Characteristics:

- ❑ Static methods do not take self or cls as their first parameter, meaning they cannot access or modify instance-specific or class-specific data.
- ❑ They can be called on the class itself or on instances of the class.
- ❑ Mentioning @staticmethod or not mentioning any decorator on method is same.

```
class MathOperations:
```

```
    @staticmethod
```

```
    def multiply(a, b):
```

```
        return a * b
```

```
*****
```

```
# Calling static methods on the class
```

```
result1 = MathOperations.multiply(3, 4)    # Output: 12
```

```
mo= MathOperations()    # can also be called on instance but no self or cls
```

```
print( mo.multiply(1,2)) ; print(result1)
```



without decorator

Direct Class Reference (Without @classmethod): You have to hard-code the class name, which is fine for simple use cases but limits flexibility if the class is extended or renamed. The method becomes less reusable in inherited classes.

inheritance **without** @classmethod

```
class Dog:
    species = "Canis familiaris"

    def species_info():
        return f"All dogs are of species {Dog.species}"
    *****

class Bulldog(Dog):
    species = "Bulldog"
    *****

print(Dog.species_info())      # Output: All dogs are of
species Canis familiaris
print(Bulldog.species_info()) # Output: All dogs are of
species Canis familiaris (method still refers to Dog)
```

inheritance **with** @classmethod

```
class Dog:
    species = "Canis familiaris"

    @classmethod
    def species_info(cls):
        return f"All dogs are of species {cls.species}"
    *****

class Bulldog(Dog):
    species = "Bulldog"
    *****

print(Dog.species_info())      # Output: All dogs are
of species Canis familiaris
print(Bulldog.species_info()) # Output: All dogs are
of species Bulldog (cls refers to subclass here)
```

Inheritance

Inheritance in Python is a mechanism that allows one class to inherit attributes and methods from another class.

Parent Class (Super Class): The class from which properties are inherited.

Child Class (Sub Class): The class that inherits from another class.

Syntax :

```
class ParentClassName:  
    pass
```

```
class ChildClassName(ParentClassName):  
    pass
```



Types of Inheritance

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Single Inheritance : A child class inherits from one parent class.

```
class Animal:  
    def speak(self):  
        print("Animal speaks")
```

```
class Dog(Animal): # Dog inherits from Animal  
    def bark(self):  
        print("Dog barks")
```

```
dog = Dog()  
dog.speak() # Inherited from Animal  
dog.bark()  # Defined in Dog
```

Types of Inheritance

Multiple Inheritance : A child class inherits from more than one parent class.

```
class Father:
```

```
    def speak(self):
```

```
        print("Father speaks")
```

```
*****
```

```
class Mother:
```

```
    def speak(self):
```

```
        print("Mother speaks")
```

```
*****
```

```
class Child(Father, Mother): # Child inherits from both Father and Mother
```

```
    pass
```

```
*****
```

```
child = Child()
```

```
child.speak() # Uses the method from Father due to method resolution order (MRO)
```

```
*****
```

ClassName.mro() or ClassName.__mro__ **# to know the order in which it calls the methods.**

Types of Inheritance

Multilevel Inheritance: A child class inherits from a parent class, which in turn inherits from another class.

```
class Grandparent:
```

```
    def greet(self):
```

```
        print("Hello from Grandparent")
```

```
*****
```

```
class Parent(Grandparent):
```

```
    def greet(self):
```

```
        print("Hello from Parent")
```

```
*****
```

```
class Child(Parent): # Inherits from Parent, which inherits from Grandparent
```

```
    pass
```

```
*****
```

```
child = Child()
```

```
child.greet() # Calls the method from Parent
```


Types of Inheritance

Hierarchical Inheritance: Multiple child classes inherit from the same parent class.

```
class Animal:
    def speak(self):
        print("Animal speaks")
*****

dog = Dog()
dog.speak() # Inherited from Animal
dog.bark()
cat = Cat()
cat.speak() # Inherited from Animal
cat.meow()
```

```
class Dog(Animal): # Dog inherits from Animal
    def bark(self):
        print("Dog barks")
*****

class Cat(Animal): # Cat also inherits from Animal
    def meow(self):
        print("Cat meows")
```



Types of Inheritance

Hybrid Inheritance: A combination of multiple inheritance types, typically a mix of single and multiple inheritance patterns.

```
class Animal:
    def speak(self):
        print("Animal speaks")
*****

# Inherits from both Mammal and Bird
class Bat(Mammal, Bird):
    def echo_location(self):
        print("Bat uses echo location")
*****

bat = Bat()
bat.speak()      # Inherited from Animal
bat.walk()       # Inherited from Mammal
bat.fly()        # Inherited from Bird
bat.echo_location() # Defined in Bat
```

```
class Mammal(Animal):
    def walk(self):
        print("Mammal walks")
*****

class Bird(Animal):
    def fly(self):
        print("Bird flies")
```

Super keyword

- The `super()` function in Python is used to call methods from a parent class (superclass) in a subclass.
- This is particularly useful in the context of inheritance.
- In multiple inheritance it following MRO.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def display_info(self):
        return f"Animal Name: {self.name}"
*****
```

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Calling the constructor of the parent class
        self.breed = breed
    def display_info(self):
        # Accessing the superclass's attribute and method
        animal_info = super().display_info() # Call the parent's display_info method
        return f"{animal_info}, Breed: {self.breed}"
```

```
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name) # Calling the constructor of the parent class
        self.color = color
    def display_info(self):
        # Accessing the superclass's attribute and method
        animal_info = super().display_info() # Call the parent's display_info method
        return f"{animal_info}, Color: {self.color}"
```

```
*****
# Creating instances of Dog and Cat
dog = Dog("Buddy", "Golden Retriever")
cat = Cat("Whiskers", "Tabby")

# Calling instance methods
print(dog.display_info()) # Output: Animal Name: Buddy, Breed: Golden Retriever
print(cat.display_info()) # Output: Animal Name: Whiskers, Color: Tabby
```



Magic or Dunder Methods

Python automatically provides these methods to all classes through auto inheritance from the object base class. Dunder is **short for "double underscore"**

`print(dir(obj))` **# Using dir() to list all methods of class, including dunder methods**

By default, if you do not override these magic methods, Python provides a default implementation.

For example, if you don't define `__str__()`, calling `str()` on an instance will return a default string (something like `<__main__.MyClass object at 0x7f9c19d3>`).

1. Initialization and Representation Methods

`__init__(self, ...)`
`__str__(self)`
`__repr__(self)`

2. Arithmetic and Comparison Methods

`__add__(self, other)`: Defines the behavior for the + operator.
`__sub__(self, other)`: Defines the behavior for the - operator.
`__mul__(self, other)`: Defines the behavior for the * operator.
`__truediv__(self, other)`: Defines the behavior for the / operator.
`__eq__(self, other)`: Defines the behavior for the equality (==) operator.
`__lt__(self, other)`: Defines the behavior for the less than (<) operator.

3. Container Emulation Methods

These methods allow custom objects to behave like containers (lists, dictionaries, etc.)

`__len__(self)`: Called by `len()`.
`__getitem__(self, key)`: Called to get an item, `obj[key]`.
`__setitem__(self, key, value)`: Called to set an item, `obj[key] = value`.
`__delitem__(self, key)`: Called to delete an item, `del obj[key]`.

4. Object Attribute Methods

These methods control access to attributes of your class.

- `__getattr__(self, name)`: Called when an attribute is not found.
- `__setattr__(self, name, value)`: Called when setting an attribute.
- `__delattr__(self, name)`: Called when deleting an attribute.

5. Context Management Methods

These methods allow you to define the behavior when entering and exiting a context (with statement).

`__enter__(self)`: Called when the context is entered.
`__exit__(self, exc_type, exc_value, traceback)`: Called when the context is exited.



Magic or Dunder Methods

Overriding dunder method:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def __str__(self):
```

```
        return f"{self.name}, {self.age} years old"
```

```
    def __repr__(self):
```

```
        return f"Person('{self.name}', {self.age})"
```

```
# Creating an object
```

```
p = Person("Alice", 30)
```

```
print(str(p)) # Output: Alice, 30 years old
```

```
print(repr(p)) # Output: Person('Alice', 30)
```

```
#to print all methods of person class including dunder  
methods
```

```
print(dir(p))
```



Access Modifiers

Access modifiers puts restrictions on accessing variables and methods directly and can prevent the accidental modification of data.

Access modifiers : Public, Private and Protected

Public Access Modifier: Theoretically, public methods and fields can be accessed directly by any class.

Protected Access Modifier: Theoretically, protected methods and fields can be accessed within the same class it is declared and its subclass. Represented with underscore `_` Ex: `_age`

Private Access Modifier: Theoretically, private methods and fields can be only accessed within the same class it is declared. Represented with Doubleunderscore `__` Ex: `__bankpin`

Note: Python do not enforce you this behavior i.e. you still can access the protected and private variables outside the class.



Access Modifiers

Python do not enforce you this behavior i.e. you still can access the protected and private variables outside the class using class name rather than instance.

Ex: classname._protectedmember classname.__privatemember

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self._name = name        # Protected variable
```

```
        self.__age = age         # Private variable
```

```
    def display_info(self):
```

```
        print(f"Name: {self._name}, Age: {self.__age}")
```

```
*****
```

```
# Creating an instance of the Person class
```

```
person = Person("Alice", 30)
```

```
# Accessing the protected variable outside the class using class name which is not a good practice
```

```
print(person._name) # Output: Alice
```

```
# Accessing the private variable outside the class (using name mangling) which is not a good practice
```

```
print(person._Person__age) # Output: 30
```



Access Modifiers

Accessing Protected attribute in child class

```
class Parent:
    def __init__(self):
        self._protected_attr = "I am a protected attribute"

    def show_protected(self):
        print(f"Parent class: {self._protected_attr}")

*****

# Create an instance of Child
child_instance = Child()

# Accessing and modifying the protected attribute in the child class
child_instance.show_protected() # Displaying inherited protected attribute

child_instance.change_protected("New protected value")
child_instance.show_protected() # Displaying modified value
```

```
class Child(Parent):
    def __init__(self):
        super().__init__()
        print(f"Child class: {self._protected_attr}") # Accessing the
        protected attribute of Parent

    def change_protected(self, new_value):
        self._protected_attr = new_value # Modifying the protected
        attribute
```




Abstraction

Abstraction allows you to **hide the internal implementation details** of a function, method, or class and expose only the necessary or relevant parts of an object.

In Python, you can achieve this using **private methods**, **properties**, or **encapsulation**, which physically hide parts of a class's implementation from outside access.

abstract classes and methods themselves **don't** directly "hide" implementation, they enforce abstraction by defining what methods a subclass must implement. They provide a structured way to create abstraction in your design.

Python uses the abc module to define abstract classes.

Abstract classes can contain both concrete (implemented) methods and abstract (unimplemented) methods.

Note: Can't instantiate abstract class, can only be inherited

Abstract class and Method

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC): # Abstract Base Class
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass # Abstract method, no implementation
```

```
    @abstractmethod
```

```
    def perimeter(self):
```

```
        pass # Abstract method, no implementation
```

```
class Rectangle(Shape):
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
    def area(self):
```

```
        return self.width * self.height # Implementing abstract method
```

```
    def perimeter(self):
```

```
        return 2 * (self.width + self.height) # Implementing abstract method
```

```
*****
```

```
# Creating an instance of Rectangle
```

```
rect = Rectangle(10, 20)
```

```
print("Area:", rect.area()) # Output: Area: 200
```

```
print("Perimeter:", rect.perimeter()) # Output: Perimeter: 60
```

```
shape = Shape() # This will raise TypeError: Can't instantiate abstract class
```



Polymorphism

"poly," meaning many, and "morph," meaning form. In essence, polymorphism allows entities (functions, objects, etc.) to take on multiple forms.

Types of Polymorphism:

Compile-Time Polymorphism (or Static Polymorphism): Achieved through **method overloading** and **operator overloading**, where the method to be executed is determined at compile time.

Run-Time Polymorphism (or Dynamic Polymorphism): Achieved through **method overriding**, where the method to be executed is determined at runtime based on the object's type.

Method Overloading

- ❑ Python does not support method overloading based on the **number or type of parameters** like some other languages
- ❑ In Python, you can define multiple methods with the same name, including constructors, but only the **last definition** will take effect.

class Dog:

```
def __init__(self): # First constructor (no parameters)
```

```
    self.name = "Unknown"
```

```
    self.age = 0
```

```
def __init__(self, name): # Second constructor (one parameter)
```

```
    self.name = name
```

```
    self.age = 0
```

```
*****
```

```
# Creating an instance of Dog
```

```
dog1 = Dog("Buddy") # Works fine
```

```
# dog2 = Dog() # This would raise an error because the first constructor is overridden
```

```
print(dog1.name) # Output: Buddy
```

Method Overloading

- Method overloading can be achieved by using **default parameters** and **Using Variable-Length Arguments**

1. default parameters

```
class Dog:
    def __init__(self, name="Unknown", age=0):
        self.name = name
        self.age = age

    def info(self):
        print(f"{self.name} is {self.age} years old.")
```

```
# Creating instances with different parameters
dog1 = Dog("Buddy", 5)
dog2 = Dog("Bella")
dog3 = Dog() # Using default parameters
```

```
dog1.info() # Output: Buddy is 5 years old.
dog2.info() # Output: Bella is 0 years old.
dog3.info() # Output: Unknown is 0 years old.
```

2. Using Variable-Length Arguments

```
class Dog:
    def __init__(self, *args):
        if len(args) == 2:
            self.name = args[0]
            self.age = args[1]
        elif len(args) == 1:
            self.name = args[0]
            self.age = 0
        else:
            self.name = "Unknown"
            self.age = 0

    def info(self):
        print(f"{self.name} is {self.age} years old.")
```

```
# Creating instances with different parameters
dog1 = Dog("Buddy", 5)
dog2 = Dog("Bella")
dog3 = Dog() # Using default values
```

```
dog1.info() # Output: Buddy is 5 years old.
dog2.info() # Output: Bella is 0 years old.
dog3.info() # Output: Unknown is 0 years old.
```

Operator Overloading

- Operator overloading allows you to define custom behavior for operators (+, -, *, etc.) for user-defined types (like classes).
- By overriding specific special methods (also called dunder methods, such as `__add__`, `__sub__`, etc.), you can change how operators behave when applied to instances of your class.

#operator overloading

class Vector:

def `__init__`(self, x, y):

self.x = x

self.y = y

def `__add__`(self, other):

Overloading the + operator

return Vector(self.x + other.x, self.y + other.y)

def `__str__`(self): # Overriding to print in a readable format

return f"Vector({self.x}, {self.y})"

Create two vectors

v1 = Vector(2, 4)

v2 = Vector(1, 3)

Add two vectors using +

v3 = v1 + v2 # This calls v1.`__add__`(v2)

print(v3) # Output: Vector(3, 7)

Method Overriding

- Inheritance allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This is known as method overriding.
- The method in the subclass will be called instead of the one in the parent class when invoked on an instance of the subclass.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self): # Overriding the speak method
        print("Dog barks")

class Cat(Animal):
    def speak(self): # Overriding the speak method
        print("Cat meows")
```

```
# Polymorphic behavior
def animal_sound(animal):
    animal.speak() # Calls the appropriate method
                  # based on the object type

dog = Dog()
cat = Cat()
animal_sound(dog) # Output: Dog barks
animal_sound(cat) # Output: Cat meows
```



Duck Typing

- Python uses **duck typing**, which is a type system based on an object's behavior rather than its explicit type.
- If an object behaves like a certain type (i.e., it implements the necessary methods), it can be used in place of that type.

In the below example both classes implemented the fly method.

Duck Typing

class Bird:

def fly(self):

return "Bird flies high in the sky!"

class Aeroplane:

def fly(self):

return "Aeroplane soars through the clouds!"

def take_off(self):

return "The airplane is taking off!"

class Fish:

def swim(self):

return "Swims in the ocean!"

```
def make_it_fly(flyable):
```

```
    print(flyable.fly())
```

```
# Creating instances of Bird and Airplane
```

```
bird = Bird()
```

```
airplane = Aeroplane()
```

```
# Both can be used interchangeably because they implement the fly method
```

```
make_it_fly(bird)    # Output: Flies high in the sky!
```

```
make_it_fly(airplane) # Output: Soars through the clouds!
```

```
# The Fish class does not implement the fly method, so it would raise an error
```

```
# Uncommenting the following line would raise an AttributeError
```

```
# fish = Fish()
```

```
# make_it_fly(fish)
```


Accessor & Mutator

- **Accessors** and **Mutators** are methods used to **get** and **set** the values of **private attributes** in a class.
- These methods are implemented using
 1. getter and setter methods
 2. @property and @<attrname>.setter decorator

Note: you cannot mention setter without getter decorator, will get an error

```
class Person:
    def __init__(self, name, age):
        self._name = name # Private attribute (by convention)
        self._age = age
    # Accessor (Getter)
    def get_name(self):
        return self._name
    # Mutator (Setter)
    def set_name(self, name):
        self._name = name
```

```
# Using accessors and mutators
person = Person("John", 25)
print(person.get_name()) # Accessor: Output "John"
Person.set_name("Jesus")
```

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age
    @property
    def name(self):
        return self._name
    @name.setter
    def name(self, value):
        self._name = value
```

```
# Using @property as getter and setter
person = Person("John", 25)
print(person.name) # Output: "John"
Person.name = "jesus"
```



Outer and Inner class

□ An **inner class** (also called a **nested class**) is a class defined inside another class

```
class Order:
```

```
    def __init__(self, order_id):
        self.order_id = order_id
        self.items = []
```

```
    def add_item(self, name, price, quantity):
        item = self.Item(name, price, quantity) # Create an item using the inner class
        self.items.append(item)
```

```
    def get_total_price(self):
        return sum([item.price * item.quantity for item in self.items])
```

```
# Inner class representing an item in the order
```

```
class Item:
```

```
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
```

```
# Example usage
```

```
order = Order("ORD1234") # Creating an order object
order.add_item("Laptop", 1500, 2) # Adding items to the order
order.add_item("Mouse", 20, 5)
```

```
order.get_total_price()
```

A large, abstract graphic consisting of several concentric, overlapping rings in shades of blue, green, and purple, framing the central text.

Thank you